INSTITUTO UNIVERSITARIO POLITÉCNICO NACIONAL

Sistema de Gestión del Servicio Comunitario

Estado de avance, decisiones técnicas y verificación del proyecto

Documento de avance para el equipo de desarrollo · 7 de los 9 módulos completados

---

1. Contexto

El Servicio Comunitario del IUNAV exige un total de **120 horas**, distribuidas en dos componentes:

- **90 horas** de trabajo directo en un proyecto comunitario
- **30 horas** de impacto social o ambiental

El proceso se llevaba con carpetas físicas y mensajería informal, lo que generaba tres problemas concretos:

Problema	Consecuencia
Horas imposibles de auditar	Nadie conoce el avance real hasta que el estudiante entrega la carpeta.
Validación tardía	El tutor descubre los errores cuando el plazo está por vencer, poniendo en riesgo el egreso.
Información dispersa	Cada rol lleva su propio registro y no existe una fuente única de verdad.

El objetivo del sistema es **una sola fuente de verdad**, con el conteo de horas automatizado y el avance visible para todos los roles en tiempo real.

2. Resolución adoptada

El punto de partida era un prototipo de una sola página con datos falsos en memoria: servía para mostrar pantallas, pero no para operar. La solución es una aplicación full-stack conectada a una base de datos real.

Capa	Tecnología	Función
Frontend	Next.js 16, React 19, Tailwind CSS 4, Radix UI, Zustand, Axios	Interfaz adaptable a móvil y escritorio
Backend	Express 4, TypeScript en modo estricto, Zod, pg	API REST, validación y reglas de negocio
Datos	PostgreSQL 17, 8 tablas	Fuente única de verdad

Tres decisiones que sostienen el diseño

1. El conteo vive en la base de datos, no en la interfaz

Las metas de 90 y 30 horas se leen de la tabla de configuración, y las horas validadas se derivan de las bitácoras con estado *aprobado*. Nadie recalcula nada a mano.

2. La validación es un flujo, no una casilla

El estudiante registra, la entrada queda como *pendiente*, el tutor aprueba, rechaza o pide corrección, y el estudiante ve el comentario. Ninguna hora suma hasta que alguien la valida.

3. Los permisos se aplican en el servidor

Cada endpoint exige un rol, y el tutor solo alcanza a ver los proyectos que tutoriza. Ocultar un botón en la interfaz no es una medida de seguridad.

3. Avance por módulos

7	1	1	4
Módulos completados	Recién entregado	Pendiente	Roles operativos

Módulo	Estado	Qué resuelve
Autenticación y enrutado protegido	Completo	Sesión con token; sin credenciales no se entra al panel.
Paneles por rol (4 variantes)	Completo	Cada rol ve su prioridad: el estudiante su progreso, el tutor su cola de trabajo, el coordinador el estado global.
Registro de usuarios	Completo	El administrador da de alta cuentas con su rol asignado.
Gestión de usuarios	Completo	Consulta y activación o desactivación de cuentas.
Bitácora comunitaria (90 h)	Completo	Carga semanal con semana, grupo y evidencia.
Servicio social y ambiental (30 h)	Completo	Registro de impacto con tipo social o ambiental.
Validación del tutor	Completo	Aprueba, rechaza o pide corrección, con comentario obligatorio.
Grupos y Proyectos	Recién	Equipos, tutores, integrantes, material y estado final por estudiante.
Configuraciones y plantillas	Pendiente	Ajuste de metas, tope diario, período activo y plantillas.

4. Detalle del módulo entregado: Grupos y Proyectos

Era una pantalla de "módulo en construcción". Ahora es un módulo completo con ocho endpoints que cubren el ciclo de vida de un proyecto:

- Crear y editar proyectos
- Asignar tutor y coordinador
- Alta y baja de integrantes con buscador
- Estado final por estudiante
- Registro de material del proyecto
- Filtros por período y por estado
- Horas acumuladas por equipo y por persona
- Lectura diferenciada para el tutor

Incluye reglas de negocio que sí afectan la integridad del conteo:

> **Reglas aplicadas**
>
> - No se puede quitar del equipo a un estudiante que ya tiene horas registradas, porque se rompería el conteo.
> - Solo se asignan estudiantes activos; un tutor o coordinador sin ese rol es rechazado.
> - El tutor no alcanza a ver ni modificar proyectos que no tutoriza.

5. Verificación

El módulo se validó contra PostgreSQL en ejecución, no solo compilado:

Prueba	Resultado
Esquema aplicado y datos de ejemplo sembrados	8 tablas, 4 proyectos, 8 integrantes, 22 registros de horas
Flujo completo por API	Crear proyecto, agregar integrantes, cambiar estados, validar
Permisos por rol, uno por uno	El tutor recibe `403` al crear; el estudiante `403` al listar
Validaciones de negocio	Integrante duplicado y estudiante con horas no removible: `409`
Validaciones de formulario	Nombre corto y URL inválida rechazados indicando el campo exacto
Compilación y análisis estático	Sin errores en servidor ni en interfaz

> **Dos errores detectados gracias a la prueba en vivo**
>
> La compilación nunca los habría mostrado. Al ejecutar el flujo real se descubrió que el guardado de proyectos con material vacío fallaba, porque el esquema exigía campos que en realidad son opcionales, y que los mensajes de validación no indicaban qué campo había fallado. Ambos quedaron corregidos.

Credenciales de prueba

Rol	Correo	Contraseña
Administrador	cmendoza@iunav.edu.ve	admin123
Coordinador	mlopez@iunav.edu.ve	coord123
Tutor	jperez@iunav.edu.ve	tutor123
Estudiante	gmartinez@iunav.edu.ve	est123

6. Trabajo pendiente

Frente	Detalle e impacto
Configuraciones y plantillas	Impide ajustar las metas de horas, el tope diario y el período activo. Es lo que más afecta al cálculo, porque hoy esos valores están fijos en la base de datos.
Subida real de archivos	Hoy el proyecto escrito y las diapositivas se registran como enlace escrito a mano, y las fotos de evidencia de las bitácoras no se suben. Requiere configurar un bucket de almacenamiento.
Firma y sello	Ya existen en el modelo de datos y aparecen como alerta en el panel del coordinador, pero no hay pantalla para cargarlos.
Versionado del código	El trabajo desarrollado no está registrado todavía en el repositorio. Es la prioridad más urgente por riesgo de pérdida.

7. Cómo levantar el proyecto en local

> **Base de datos** (PostgreSQL 17, base `app_iunav`)
>
> En `server/.env`: `DATABASE_URL=postgresql://usuario:clave@localhost:5432/app_iunav` y `DB_SSL=false`
>
> **Instalación y arranque**
>
> `cd server` → `npm run db:schema` → `npm run db:seed` → `npm run dev`
> `cd web` → `npm run dev` → abrir `http://localhost:3000`